

Praneesh R V
CB.SC.U4CYS23036

Raft Implementation

```
x ▶ cd Praneesh/Academics/Sem7/DSCC/
0s ~ .. /Academics/Sem7/DSCC ~ clean-push:main
• ▶ python raft_single.py node \
    --id n1 \
    --bind 127.0.0.1:9001 \
    --peer n2=127.0.0.1:9002 \
    --peer n3=127.0.0.1:9003
2026-08-10 10:27:24,279 INFO [n1] listening on ('127.0.0.1', 9001)
2026-08-10 10:27:25,171 INFO [n1] starting election for term 1
2026-08-10 10:27:26,344 INFO [n1] starting election for term 2
2026-08-10 10:27:27,440 INFO [n1] starting election for term 3
2026-08-10 10:27:28,532 INFO [n1] starting election for term 4
2026-08-10 10:27:29,550 INFO [n1] starting election for term 5
2026-08-10 10:27:30,924 INFO [n1] starting election for term 6
2026-08-10 10:27:31,791 INFO [n1] starting election for term 7
2026-08-10 10:27:32,986 INFO [n1] starting election for term 8
2026-08-10 10:27:32,995 INFO [n1] became LEADER for term 8
|
```

```
0s ~    
• ▶ cd Praneesh/Academics/Sem7/DSCC/  
0s ~   ../Academics/Sem7/DSCC ~  clean-push:main  
• ▶ python raft_single.py node \  
    --id n2 \  
    --bind 127.0.0.1:9002 \  
    --peer n1=127.0.0.1:9001 \  
    --peer n3=127.0.0.1:9003  
2026-08-10 10:27:31,944 INFO [n2] listening on ('127.0.0.1', 9002)  
|
```

```
0s ~    
• ▶ cd Praneesh/Academics/Sem7/DSCC/  
0s ~   ../Academics/Sem7/DSCC ~  clean-push:main  
• ▶ python raft_single.py node \  
    --id n3 \  
    --bind 127.0.0.1:9003 \  
    --peer n1=127.0.0.1:9001 \  
    --peer n2=127.0.0.1:9002  
2026-08-10 10:27:41,350 INFO [n3] listening on ('127.0.0.1', 9003)  
|
```

```
43s ~ .. /Academics/Sem7/DSCC ~ P clean-push:main
• ▶ python raft_single.py client --address 127.0.0.1:9001 status
{
  "ok": true,
  "node_id": "n1",
  "role": "leader",
  "term": 8,
  "leader_id": "n1",
  "leader_address": "127.0.0.1:9001",
  "commit_index": 0,
  "last_applied": 0,
  "last_log_index": 0,
  "last_log_term": 8,
  "log_length": 1,
  "kv_items": 0,
  "peers": {
    "n2": "127.0.0.1:9002",
    "n3": "127.0.0.1:9003"
  }
}
0s ~ .. /Academics/Sem7/DSCC ~ P clean-push:main
• ▶ |
```

```
#!/usr/bin/env python3
```

```
from __future__ import annotations
```

```
import argparse
```

```
import asyncio
```

```
import json
```

```
import logging
```

```
import os
```

```
import random
```

```
import signal
```

```
import time
```

```
from dataclasses import dataclass
```

```
from pathlib import Path
```

```
from typing import Any, Dict, Optional, Tuple
```

```
LOG = logging.getLogger("raft")
```

```

@dataclass
class RaftConfig:
    node_id: str
    bind_host: str
    bind_port: int
    peers: Dict[str, Tuple[str, int]]
    data_dir: Path
    heartbeat_interval: float = 0.20
    election_timeout_min: float = 0.80
    election_timeout_max: float = 1.50
    rpc_timeout: float = 0.50

```

```

class RaftNode:
    FOLLOWER = "follower"
    CANDIDATE = "candidate"
    LEADER = "leader"

    def __init__(self, config: RaftConfig) -> None:
        self.cfg = config
        self.node_id = config.node_id
        self.peers = config.peers
        self.majority = (len(self.peers) + 1) // 2 + 1

        # Persistent Raft state.
        self.current_term: int = 0
        self.voted_for: Optional[str] = None
        self.log: list[dict[str, Any]] = []
        self.commit_index: int = -1

        # Volatile state.
        self.role = self.FOLLOWER
        self.leader_id: Optional[str] = None
        self.last_applied: int = -1
        self.state_machine: Dict[str, Any] = {}

        # Leader-only volatile state.
        self.next_index: Dict[str, int] = {}
        self.match_index: Dict[str, int] = {}

        self._state_lock = asyncio.Lock()
        self._peer_locks = {peer_id: asyncio.Lock() for peer_id in self.peers}
        self._server: Optional[asyncio.AbstractServer] = None

```

```

self._tasks: list[asyncio.Task[Any]] = []
self._stopping = asyncio.Event()
self._election_deadline = 0.0

self.cfg.data_dir.mkdir(parents=True, exist_ok=True)
self._state_path = self.cfg.data_dir / f"{self.node_id}.json"
self._load_state()
self._reset_election_deadline()

# -----
# Persistence + state machine
# -----

def _load_state(self) -> None:
    if not self._state_path.exists():
        return

    try:
        data = json.loads(self._state_path.read_text(encoding="utf-8"))
        self.current_term = int(data.get("current_term", 0))
        self.voted_for = data.get("voted_for")
        self.log = list(data.get("log", []))
        self.commit_index = min(int(data.get("commit_index", -1)), len(self.log) - 1)
        self._apply_committed_locked()
        LOG.info(
            "[%s] recovered term=%s log=%s commit=%s",
            self.node_id,
            self.current_term,
            len(self.log),
            self.commit_index,
        )
    except Exception:
        LOG.exception("[%s] failed to read persistent state", self.node_id)
        raise

def _persist_locked(self) -> None:
    """Persist Raft state atomically. Caller must hold _state_lock."""
    data = {
        "current_term": self.current_term,
        "voted_for": self.voted_for,
        "log": self.log,
        "commit_index": self.commit_index,
    }
    tmp = self._state_path.with_suffix(".tmp")

```

```

encoded = json.dumps(data, separators=(",", ":"), ensure_ascii=False)

with open(tmp, "w", encoding="utf-8") as f:
    f.write(encoded)
    f.flush()
    os.fsync(f.fileno())
os.replace(tmp, self._state_path)

def _apply_committed_locked(self) -> None:
    while self.last_applied < self.commit_index:
        self.last_applied += 1
        command = self.log[self.last_applied]["command"]
        op = command.get("op")

        if op == "set":
            self.state_machine[str(command["key"])] = command.get("value")
        elif op == "delete":
            self.state_machine.pop(str(command["key"]), None)
        elif op == "noop":
            pass
        else:
            LOG.warning(
                "[%s] ignoring unknown committed command at index %s: %r",
                self.node_id,
                self.last_applied,
                command,
            )

# -----
# Small helpers
# -----

def _reset_election_deadline(self) -> None:
    self._election_deadline = time.monotonic() + random.uniform(
        self.cfg.election_timeout_min,
        self.cfg.election_timeout_max,
    )

def _last_log_info_locked(self) -> tuple[int, int]:
    if not self.log:
        return -1, 0
    return len(self.log) - 1, int(self.log[-1]["term"])

def _leader_address_locked(self) -> Optional[str]:

```

```

    if self.leader_id == self.node_id:
        return f"{self.cfg.bind_host}:{self.cfg.bind_port}"
    if self.leader_id and self.leader_id in self.peers:
        host, port = self.peers[self.leader_id]
        return f"{host}:{port}"
    return None

def _step_down_locked(self, new_term: int, leader_id: Optional[str] = None) -> None:
    term_changed = new_term > self.current_term
    if term_changed:
        self.current_term = new_term
        self.voted_for = None

    self.role = self.FOLLOWER
    self.leader_id = leader_id
    self.next_index.clear()
    self.match_index.clear()
    self._reset_election_deadline()

    if term_changed:
        self._persist_locked()

# -----
# Networking
# -----

async def start(self) -> None:
    self._server = await asyncio.start_server(
        self._handle_connection,
        self.cfg.bind_host,
        self.cfg.bind_port,
    )

    sockets = self._server.sockets or []
    bound = ", ".join(str(s.getsockname()) for s in sockets)
    LOG.info("[%s] listening on %s", self.node_id, bound)

    self._tasks = [
        asyncio.create_task(self._election_loop(), name=f"{self.node_id}-election"),
        asyncio.create_task(self._heartbeat_loop(), name=f"{self.node_id}-heartbeat"),
    ]

async def stop(self) -> None:
    if self._stopping.is_set():

```

```

        return
    self._stopping.set()

    if self._server is not None:
        self._server.close()
        await self._server.wait_closed()

    for task in self._tasks:
        task.cancel()
    await asyncio.gather(*self._tasks, return_exceptions=True)

    async def _handle_connection(
        self,
        reader: asyncio.StreamReader,
        writer: asyncio.StreamWriter,
    ) -> None:
        try:
            raw = await asyncio.wait_for(reader.readline(), timeout=5.0)
            if not raw:
                return

            request = json.loads(raw.decode("utf-8"))
            response = await self._dispatch(request)
        except asyncio.TimeoutError:
            response = {"ok": False, "error": "request_timeout"}
        except Exception as exc:
            LOG.exception("[%s] request handler failed", self.node_id)
            response = {"ok": False, "error": f"bad_request: {exc}"}

        try:
            writer.write((json.dumps(response, separators=(",", ":")) + "\n").encode("utf-8"))
            await writer.drain()
        finally:
            writer.close()
            await writer.wait_closed()

    async def _dispatch(self, msg: dict[str, Any]) -> dict[str, Any]:
        msg_type = msg.get("type")
        if msg_type == "request_vote":
            return await self._on_request_vote(msg)
        if msg_type == "append_entries":
            return await self._on_append_entries(msg)
        if msg_type == "client_set":
            return await self._on_client_set(msg)

```

```

    if msg_type == "client_delete":
        return await self._on_client_delete(msg)
    if msg_type == "client_get":
        return await self._on_client_get(msg)
    if msg_type == "status":
        return await self._on_status()
    return {"ok": False, "error": f"unknown_message_type: {msg_type!r}"}

async def _rpc(
    self,
    address: tuple[str, int],
    message: dict[str, Any],
) -> Optional[dict[str, Any]]:
    host, port = address
    try:
        reader, writer = await asyncio.wait_for(
            asyncio.open_connection(host, port),
            timeout=self.cfg.rpc_timeout,
        )
        writer.write((json.dumps(message, separators=(",", ":")) + "\n").encode("utf-8"))
        await writer.drain()
        raw = await asyncio.wait_for(reader.readline(), timeout=self.cfg.rpc_timeout)
        writer.close()
        await writer.wait_closed()
        if not raw:
            return None
        return json.loads(raw.decode("utf-8"))
    except (OSError, asyncio.TimeoutError, json.JSONDecodeError):
        return None

# -----
# Elections
# -----

async def _election_loop(self) -> None:
    while not self._stopping.is_set():
        await asyncio.sleep(0.025)

    async with self._state_lock:
        should_start = (
            self.role != self.LEADER
            and time.monotonic() >= self._election_deadline
        )

```

```

        if should_start:
            await self._start_election()

    async def _start_election(self) -> None:
        async with self._state_lock:
            self.role = self.CANDIDATE
            self.leader_id = None
            self.current_term += 1
            self.voted_for = self.node_id
            self._reset_election_deadline()
            self._persist_locked()

            term = self.current_term
            last_index, last_term = self._last_log_info_locked()

        LOG.info("[%s] starting election for term %s", self.node_id, term)

        if self.majority == 1:
            async with self._state_lock:
                if self.role == self.CANDIDATE and self.current_term == term:
                    await self._become_leader_locked()
            return

        request = {
            "type": "request_vote",
            "term": term,
            "candidate_id": self.node_id,
            "last_log_index": last_index,
            "last_log_term": last_term,
        }

        async def ask(peer_id: str, address: tuple[str, int]) -> tuple[str, Optional[dict[str, Any]]]:
            return peer_id, await self._rpc(address, request)

        tasks = [asyncio.create_task(ask(pid, addr)) for pid, addr in self.peers.items()]
        votes = 1

        try:
            for future in asyncio.as_completed(tasks):
                peer_id, response = await future
                if response is None:
                    continue

            async with self._state_lock:

```

```

        response_term = int(response.get("term", 0))
        if response_term > self.current_term:
            self._step_down_locked(response_term)
            LOG.info(
                "[%s] election abandoned: %s has newer term %s",
                self.node_id,
                peer_id,
                response_term,
            )
            return

        if self.current_term != term or self.role != self.CANDIDATE:
            return

        if response.get("vote_granted"):
            votes += 1
            if votes >= self.majority:
                await self._become_leader_locked()
                return

    finally:
        for task in tasks:
            if not task.done():
                task.cancel()
        await asyncio.gather(*tasks, return_exceptions=True)

async def _become_leader_locked(self) -> None:
    """Caller must hold _state_lock."""
    self.role = self.LEADER
    self.leader_id = self.node_id
    self.next_index = {peer_id: len(self.log) for peer_id in self.peers}
    self.match_index = {peer_id: -1 for peer_id in self.peers}

    # A no-op in the leader's current term helps establish leadership and
    # allows Raft's current-term commit rule to commit older entries safely.
    self.log.append({"term": self.current_term, "command": {"op": "noop"}})
    self._persist_locked()
    self._advance_commit_index_locked()

    LOG.info("[%s] became LEADER for term %s", self.node_id, self.current_term)
    asyncio.create_task(self._replicate_all())

async def _on_request_vote(self, msg: dict[str, Any]) -> dict[str, Any]:
    term = int(msg["term"])
    candidate_id = str(msg["candidate_id"])

```

```
candidate_last_index = int(msg["last_log_index"])
candidate_last_term = int(msg["last_log_term"])
```

```
async with self._state_lock:
    if term < self.current_term:
        return {"term": self.current_term, "vote_granted": False}

    if term > self.current_term:
        self._step_down_locked(term)
```

```
my_last_index, my_last_term = self._last_log_info_locked()
candidate_is_up_to_date = (
    candidate_last_term > my_last_term
    or (
        candidate_last_term == my_last_term
        and candidate_last_index >= my_last_index
    )
)
```

```
can_vote = self.voted_for is None or self.voted_for == candidate_id
if can_vote and candidate_is_up_to_date:
    self.voted_for = candidate_id
    self._reset_election_deadline()
    self._persist_locked()
    return {"term": self.current_term, "vote_granted": True}
```

```
return {"term": self.current_term, "vote_granted": False}
```

```
# -----
# AppendEntries / replication
# -----
```

```
async def _heartbeat_loop(self) -> None:
    while not self._stopping.is_set():
        await asyncio.sleep(self.cfg.heartbeat_interval)
        async with self._state_lock:
            is_leader = self.role == self.LEADER
        if is_leader:
            await self._replicate_all()
```

```
async def _replicate_all(self) -> None:
    async with self._state_lock:
        if self.role != self.LEADER:
            return
```

[illegible]

```

        return

    if self.role != self.LEADER or self.current_term != term:
        return

    if response.get("success"):
        match = int(response.get("match_index", prev_idx + len(entries)))
        self.match_index[peer_id] = max(self.match_index.get(peer_id, -1), match)
        self.next_index[peer_id] = self.match_index[peer_id] + 1
        self._advance_commit_index_locked()
        return

    conflict_index = response.get("conflict_index")
    if conflict_index is not None:
        self.next_index[peer_id] = max(0, int(conflict_index))
    else:
        self.next_index[peer_id] = max(0, next_idx - 1)

def _advance_commit_index_locked(self) -> None:
    """Leader commit advancement. Caller must hold _state_lock."""
    if self.role != self.LEADER:
        return

    for index in range(len(self.log) - 1, self.commit_index, -1):
        # Raft only advances commitIndex by counting replicas for an entry
        # from the leader's current term.
        if int(self.log[index]["term"]) != self.current_term:
            continue

        replicated = 1 # leader itself
        replicated += sum(1 for m in self.match_index.values() if m >= index)

        if replicated >= self.majority:
            self.commit_index = index
            self._apply_committed_locked()
            self._persist_locked()
            break

async def _on_append_entries(self, msg: dict[str, Any]) -> dict[str, Any]:
    term = int(msg["term"])
    leader_id = str(msg["leader_id"])
    prev_idx = int(msg["prev_log_index"])
    prev_term = int(msg["prev_log_term"])
    entries = list(msg.get("entries", []))

```

```

leader_commit = int(msg.get("leader_commit", -1))

async with self._state_lock:
    if term < self.current_term:
        return {
            "term": self.current_term,
            "success": False,
            "conflict_index": len(self.log),
        }

    if term > self.current_term:
        self._step_down_locked(term, leader_id=leader_id)
    else:
        # Valid AppendEntries from the current-term leader forces a
        # candidate/old leader back to follower state.
        if self.role != self.FOLLOWER:
            self.role = self.FOLLOWER
            self.next_index.clear()
            self.match_index.clear()
        self.leader_id = leader_id
        self._reset_election_deadline()

    if prev_idx >= len(self.log):
        return {
            "term": self.current_term,
            "success": False,
            "conflict_index": len(self.log),
        }

    if prev_idx >= 0 and int(self.log[prev_idx]["term"]) != prev_term:
        bad_term = int(self.log[prev_idx]["term"])
        first = prev_idx
        while first > 0 and int(self.log[first - 1]["term"]) == bad_term:
            first -= 1
        return {
            "term": self.current_term,
            "success": False,
            "conflict_index": first,
        }

    changed = False
    insert_at = prev_idx + 1
    i = 0

```

```

while i < len(entries) and insert_at + i < len(self.log):
    local = self.log[insert_at + i]
    incoming = entries[i]
    if int(local["term"]) != int(incoming["term"]):
        self.log = self.log[: insert_at + i]
        changed = True
        break
    i += 1

if i < len(entries):
    self.log.extend(entries[i:])
    changed = True

if leader_commit > self.commit_index:
    self.commit_index = min(leader_commit, len(self.log) - 1)
    self._apply_committed_locked()
    changed = True

if changed:
    self._persist_locked()

match_index = prev_idx + len(entries)
return {
    "term": self.current_term,
    "success": True,
    "match_index": match_index,
}

# -----
# Client API
# -----

async def _not_leader_response(self) -> dict[str, Any]:
    async with self._state_lock:
        return {
            "ok": False,
            "error": "not_leader",
            "leader_id": self.leader_id,
            "leader_address": self._leader_address_locked(),
            "term": self.current_term,
        }

async def _submit_command(
    self,

```

```

command: dict[str, Any],
timeout: float = 5.0,
) -> tuple[bool, dict[str, Any]]:
    async with self._state_lock:
        if self.role != self.LEADER:
            return False, {
                "ok": False,
                "error": "not_leader",
                "leader_id": self.leader_id,
                "leader_address": self._leader_address_locked(),
                "term": self.current_term,
            }

        term = self.current_term
        self.log.append({"term": term, "command": command})
        index = len(self.log) - 1
        self._persist_locked()
        self._advance_commit_index_locked()

        asyncio.create_task(self._replicate_all())
        deadline = time.monotonic() + timeout

        while time.monotonic() < deadline:
            async with self._state_lock:
                if self.commit_index >= index:
                    return True, {
                        "ok": True,
                        "index": index,
                        "term": term,
                        "leader_id": self.node_id,
                    }

            if self.role != self.LEADER or self.current_term != term:
                return False, {
                    "ok": False,
                    "error": "leadership_lost",
                    "leader_id": self.leader_id,
                    "leader_address": self._leader_address_locked(),
                    "term": self.current_term,
                }

            await asyncio.sleep(0.02)

    return False, {

```

```

    "ok": False,
    "error": "commit_timeout",
    "index": index,
    "term": term,
}

```

```

async def _on_client_set(self, msg: dict[str, Any]) -> dict[str, Any]:
    if "key" not in msg:
        return {"ok": False, "error": "missing_key"}
    command = {"op": "set", "key": str(msg["key"]), "value": msg.get("value")}
    _, response = await self._submit_command(command)
    return response

```

```

async def _on_client_delete(self, msg: dict[str, Any]) -> dict[str, Any]:
    if "key" not in msg:
        return {"ok": False, "error": "missing_key"}
    command = {"op": "delete", "key": str(msg["key"])}
    _, response = await self._submit_command(command)
    return response

```

```

async def _on_client_get(self, msg: dict[str, Any]) -> dict[str, Any]:
    if "key" not in msg:
        return {"ok": False, "error": "missing_key"}

```

```

    # Committing a no-op first acts as a simple read barrier. It is not the
    # most efficient technique, but it gives a straightforward linearizable
    # read without implementing Raft ReadIndex/leases.
    committed, response = await self._submit_command({"op": "noop"})
    if not committed:
        return response

```

```

    key = str(msg["key"])
    async with self._state_lock:
        return {
            "ok": True,
            "key": key,
            "found": key in self.state_machine,
            "value": self.state_machine.get(key),
            "term": self.current_term,
            "leader_id": self.node_id,
        }

```

```

async def _on_status(self) -> dict[str, Any]:
    async with self._state_lock:

```

```

last_idx, last_term = self._last_log_info_locked()
return {
    "ok": True,
    "node_id": self.node_id,
    "role": self.role,
    "term": self.current_term,
    "leader_id": self.leader_id,
    "leader_address": self._leader_address_locked(),
    "commit_index": self.commit_index,
    "last_applied": self.last_applied,
    "last_log_index": last_idx,
    "last_log_term": last_term,
    "log_length": len(self.log),
    "kv_items": len(self.state_machine),
    "peers": {
        peer_id: f"{addr[0]}:{addr[1]}"
        for peer_id, addr in self.peers.items()
    },
}

```

```

# -----
# CLI helpers
# -----

```

```

def parse_address(value: str) -> tuple[str, int]:
    try:
        host, port_text = value.rsplit(":", 1)
        if not host:
            raise ValueError
        port = int(port_text)
        if not (1 <= port <= 65535):
            raise ValueError
        return host, port
    except ValueError as exc:
        raise argparse.ArgumentTypeError(
            f"invalid address {value!r}; expected HOST:PORT"
        ) from exc

```

```

def parse_peer(value: str) -> tuple[str, tuple[str, int]]:
    try:
        peer_id, address = value.split("=", 1)

```

```

    if not peer_id:
        raise ValueError
    return peer_id, parse_address(address)
except ValueError as exc:
    raise argparse.ArgumentTypeError(
        f"invalid peer {value!r}; expected ID=HOST:PORT"
    ) from exc

```

```

def parse_jsonish(value: str) -> Any:
    """Interpret CLI values as JSON when possible, otherwise keep as string."""
    try:
        return json.loads(value)
    except json.JSONDecodeError:
        return value

```

```

async def run_node(args: argparse.Namespace) -> None:
    peers = dict(args.peer)
    if args.id in peers:
        raise SystemExit("--peer must not contain this node's own id")

```

```

bind_host, bind_port = args.bind
cfg = RaftConfig(
    node_id=args.id,
    bind_host=bind_host,
    bind_port=bind_port,
    peers=peers,
    data_dir=Path(args.data_dir),
    heartbeat_interval=args.heartbeat,
    election_timeout_min=args.election_min,
    election_timeout_max=args.election_max,
    rpc_timeout=args.rpc_timeout,
)

```

```

if cfg.election_timeout_min <= cfg.heartbeat_interval * 2:
    raise SystemExit("election timeout should be comfortably larger than heartbeat interval")
if cfg.election_timeout_max <= cfg.election_timeout_min:
    raise SystemExit("--election-max must be greater than --election-min")

```

```

node = RaftNode(cfg)
await node.start()

```

```

stop_event = asyncio.Event()

```

```
loop = asyncio.get_running_loop()
```

```
def request_stop() -> None:  
    stop_event.set()
```

```
for sig in (signal.SIGINT, signal.SIGTERM):  
    try:  
        loop.add_signal_handler(sig, request_stop)  
    except NotImplementedError:  
        pass
```

```
try:  
    await stop_event.wait()  
finally:  
    await node.stop()
```

```
async def client_rpc(address: tuple[str, int], message: dict[str, Any]) -> dict[str, Any]:  
    host, port = address  
    reader, writer = await asyncio.open_connection(host, port)  
    try:  
        writer.write((json.dumps(message, separators=(",", ":")) + "\n").encode("utf-8"))  
        await writer.drain()  
        raw = await asyncio.wait_for(reader.readline(), timeout=6.0)  
        if not raw:  
            raise RuntimeError("server closed connection without a response")  
        return json.loads(raw.decode("utf-8"))  
    finally:  
        writer.close()  
        await writer.wait_closed()
```

```
async def run_client(args: argparse.Namespace) -> None:  
    if args.command == "status":  
        message = {"type": "status"}  
    elif args.command == "set":  
        message = {  
            "type": "client_set",  
            "key": args.key,  
            "value": parse_jsonish(args.value),  
        }  
    elif args.command == "get":  
        message = {"type": "client_get", "key": args.key}  
    elif args.command == "delete":
```

```
    message = {"type": "client_delete", "key": args.key}
else:
    raise RuntimeError(f"unknown command: {args.command}")
```

```
response = await client_rpc(args.address, message)
print(json.dumps(response, indent=2, ensure_ascii=False))
```

```
def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(description="Single-file Raft implementation")
    parser.add_argument(
        "--log-level",
        default="INFO",
        choices=["DEBUG", "INFO", "WARNING", "ERROR"],
    )

    sub = parser.add_subparsers(dest="mode", required=True)

    node = sub.add_parser("node", help="start a Raft node")
    node.add_argument("--id", required=True, help="unique node id, e.g. n1")
    node.add_argument("--bind", required=True, type=parse_address, help="HOST:PORT")
    node.add_argument(
        "--peer",
        action="append",
        type=parse_peer,
        default=[],
        metavar="ID=HOST:PORT",
        help="peer node; repeat once per peer",
    )
    node.add_argument("--data-dir", default="./raft-data")
    node.add_argument("--heartbeat", type=float, default=0.20)
    node.add_argument("--election-min", type=float, default=0.80)
    node.add_argument("--election-max", type=float, default=1.50)
    node.add_argument("--rpc-timeout", type=float, default=0.50)

    client = sub.add_parser("client", help="send a client command")
    client.add_argument("--address", required=True, type=parse_address, help="HOST:PORT")
    client_sub = client.add_subparsers(dest="command", required=True)

    client_sub.add_parser("status")

    set_cmd = client_sub.add_parser("set")
    set_cmd.add_argument("key")
    set_cmd.add_argument("value")
```

```
get_cmd = client_sub.add_parser("get")
get_cmd.add_argument("key")

delete_cmd = client_sub.add_parser("delete")
delete_cmd.add_argument("key")

return parser
```

```
def main() -> None:
    parser = build_parser()
    args = parser.parse_args()

    logging.basicConfig(
        level=getattr(logging, args.log_level),
        format="%(asctime)s %(levelname)s %(message)s",
    )

    try:
        if args.mode == "node":
            asyncio.run(run_node(args))
        else:
            asyncio.run(run_client(args))
    except KeyboardInterrupt:
        pass

if __name__ == "__main__":
    main()
```